Hochschule Bremen
Faculty 4 – Electrical Engineering and Computer Science

Concept

ESP32-based PV surplus load controller with local user interface and Home Assistant integration

Name: Enrico Brodtmann
Student ID: 5265217
Degree Program: Media Informatics B.Sc.
Module: Microcontrollers and Applications
Date: 17.04.2026

1 Project Idea

The aim of this project is to develop a microcontroller-based control unit that can automatically switch high-power consumers depending on the available photovoltaic surplus.

The central control unit shall be an ESP32. The main logic of the system shall run locally on the ESP32 so that the controller remains operational even without Home Assistant or Node-RED. Home Assistant and/or Node-RED shall only serve as optional external interfaces for visualization, monitoring, and parameter adjustment.

2 Purpose of the System

In households with a large PV system, surplus electrical energy is temporarily available. This project shall provide a prototype that can automatically activate or deactivate selected loads in order to make efficient use of this surplus.

Possible loads include, for example, a heating element or another large consumer such as a miner. Since such loads exceed the direct switching capability of a microcontroller by far, the ESP32 shall only control an external contactor or relay via a low-voltage control path.

3 Functional Requirements

The system shall fulfill the following functions:

- measure or receive a control value representing the currently available PV surplus,
- make a local decision on the ESP32 whether a load should be switched on or off,
- provide at least one switching output for controlling an external contactor,
- support different operating modes such as *OFF*, *MANUAL*, and *AUTO*,
- provide a local tactile user interface with physical buttons, switches, or a rotary encoder,
- provide local status indication via LEDs and/or a small display,
- transmit status information via WiFi to Home Assistant or Node-RED,
- allow manual override through the local user interface.

This matches the course requirement that the device must provide a physical, tactile user interface and must not be operated exclusively via smartphone or web interface.

4 Technical Concept

The system is based on an ESP32 because this platform offers WiFi, sufficient processing power, and versatile input and output options for embedded control tasks. The ESP32 is particularly suitable for projects in which local control logic and network connectivity need to be combined.

The planned hardware components are:

- an ESP32 development board,
- buttons or a rotary encoder for local input,
- LEDs and/or an OLED display for local output,
- a low-voltage switching stage with transistor, optocoupler, or relay module,
- an external contactor for the actual high-power consumer,
- optionally a sensor or data interface for acquiring surplus information,
- a WiFi connection for integration into Home Assistant or Node-RED.

5 Software Concept

The ESP32 firmware shall implement the complete control logic locally. This includes in particular the following tasks:

- reading buttons and local settings,
- evaluating surplus information,
- applying switching rules with hysteresis and safety delays,
- controlling the switching output,
- sending status values via WiFi,
- receiving optional external commands or configuration data.

An important architectural goal is that Home Assistant can monitor and configure the controller, while the basic functionality of the system remains available even in case of network failure or failure of the external platform.

6 User Interface

The device shall include a local physical user interface. Planned elements include, for example:

- a selector switch or button for the operating modes *AUTO*, *MANUAL*, and *OFF*,
- a manual ON/OFF button for the load,
- status LEDs for power supply, WiFi connection, and load state,
- optionally a small display showing mode, surplus state, and switching status.

Physical operability is a central part of the project because the course explicitly requires a real, tactile user interface.

7 Scope and Delimitation

The project is intended as a proof of concept and not as a certified, production-ready high-power control device. Therefore, the focus is on:

- a correctly functioning local decision-making logic,
- safe low-voltage control of an external switching element,
- local operability,
- WiFi-based monitoring and integration.

A complete industrial-grade mains power installation is not part of the project. This limitation is also consistent with the course note that a functional prototype or proof of concept is sufficient.

8 Expected Result

At the end of the project, a functional prototype shall demonstrate that an ESP32-based controller is capable of autonomously managing a high-power consumer on the basis of available PV surplus power, while remaining locally operable and being integrable into Home Assistant.

9 Final Expansion Stage

The final expansion stage of the project aims at making the controller fully independent of any external control system. While the basic prototype can already operate locally without Home Assistant or Node-RED, the surplus information in the basic stage is still derived from an external data source such as a smart meter or an MQTT message provided by another system. In the final expansion stage, this dependency is eliminated by integrating the measurement directly into the device itself.

9.1 Direct three-phase measurement

For the final expansion stage, three dedicated measurement channels shall be added to the controller, one for each phase of the household feed. The intended reference is a measurement module comparable to a Shelly Pro 3EM, which acquires for each phase:

- the line voltage,
- the current via a current transformer (CT clamp),
- the active, apparent and reactive power derived from these,
- the direction of the power flow (import from grid vs. export to grid).

By measuring all three phases, the controller can determine the actual PV surplus by itself, without relying on data from an external smart meter, an inverter API, or an external home automation system.

9.2 Standalone operation

With the integrated three-phase measurement, the system becomes a true standalone device. All steps of the control loop – measurement, decision, switching, and status indication – are performed locally on the ESP32. The external infrastructure (Home Assistant, Node-RED, MQTT broker, internet connection) is then only used for optional visualization, logging, and remote configuration. The controller remains fully functional even if every external system is unavailable.

This is a significant property for two reasons. First, it removes a single point of failure: a defective home server or a network outage cannot disable the surplus management. Second, it allows the device to be deployed in installations where no home automation infrastructure exists at all.

9.3 Battery-backed operation

Because all relevant control and measurement components operate on low DC voltage, the entire low-voltage section of the controller (ESP32, measurement front end, driver stage logic) can optionally be supplied from a small battery or an uninterruptible power source. This enables two scenarios:

- *Bridging short power outages:* the controller continues to monitor the situation and re-engages the load in a defined manner once mains power returns,
- *Off-grid or island-grid operation:* the controller can be used in setups where the supply is provided by a battery system rather than a public grid.

The high-power switching element (contactor) naturally still requires its own coil supply and the load itself still requires the AC mains, but the decision-making part of the system is no longer tied to the availability of the main supply.

9.4 Resulting system properties

The final expansion stage therefore delivers a controller that is:

- self-measuring (three-phase voltage, current, power, direction of flow),
- self-deciding (local control logic on the ESP32),
- self-sufficient (no external system required for normal operation),
- optionally battery-backed for continued operation during supply interruptions.

This expansion stage is intentionally described as a goal for the final version of the prototype. The basic prototype delivered within the scope of the course already implements the local control logic and the safe switching path; the integration of the dedicated three-phase measurement and the optional battery backup are presented here as the planned next development step.

10 Test Sequence and Verification Scenarios

In order to systematically verify that the prototype fulfills its intended purpose, a structured test sequence is defined. The verification is organized along three primary goals. Each goal is broken down into individual test cases consisting of a precondition, a test action, and an expected result. The tests are designed to be executed in increasing order of complexity, so that each subsequent test relies on functionality that has already been verified in the previous step.

10.1 Goal 1: Switching of an AC load via low-voltage DC control

The first goal is to verify that the ESP32 can safely switch a higher AC voltage by means of a low-voltage DC control signal. The galvanic separation between the microcontroller and the AC side is the central safety property to be confirmed.

Test 1.1 – Control signal generation. With the ESP32 powered and idle, the firmware is commanded to set the switching output to logic HIGH. Using a multimeter, the voltage at the GPIO pin is measured. The expected result is a stable level of approximately 3.3 V.

Test 1.2 – Driver stage and relay actuation. The GPIO output is connected to the input of the driver stage (transistor or optocoupler) which controls a relay or solid-state relay. When the firmware toggles the output, an audible click of the relay (or activation of the SSR status LED) shall be observed. The continuity of the relay contacts is verified with a multimeter in both states.

Test 1.3 – Switching a real AC load (single-phase). A small single-phase AC load (for example a 230 V incandescent lamp) is connected through the relay. The firmware is commanded to switch the output ON and OFF in defined intervals. The expected result is that the lamp follows the commanded state reliably and that no abnormal heating of the driver stage occurs.

Test 1.4 – Galvanic separation check. With the AC side energized and the load switched, the insulation between the ESP32 ground and the AC conductors is verified. The expected result is that no measurable coupling exists between the low-voltage control side and the AC side.

10.2 Goal 2: Switching of large three-phase loads (400 V) via the microcontroller

The second goal extends the verified switching path of Goal 1 to a real three-phase load at 400 V. Since the ESP32 itself never carries the load current, this is achieved by using the already verified low-voltage control path to actuate a three-phase contactor.

Test 2.1 – Contactor coil actuation. The output of the relay verified in Test 1.2 is wired to the coil of a three-phase contactor. When the ESP32 issues a switching command, the contactor shall pick up audibly and its auxiliary contact shall close. This is verified with a multimeter on the auxiliary contact.

Test 2.2 – Three-phase load switching (resistive). With the contactor wired to a three-phase resistive load (for example a heating element), the firmware switches the contactor ON. Each of the three phases is checked for voltage continuity to the load using a category-rated multimeter. The expected result is that all three phases

are switched simultaneously and that the load operates as expected.

Test 2.3 – Repeated switching cycles. The firmware performs a defined number of switching cycles (for example 20 cycles with a dwell time of several seconds) under load. The expected result is that all switching events are executed without contact welding, without unintended re-triggering, and without observable temperature rise of the control electronics.

Test 2.4 – Behavior on power loss of the ESP32. While the load is switched ON, the supply of the ESP32 is intentionally interrupted. The expected result is that the contactor drops out and the load is de-energized within the response time of the relay, since the system is designed to be fail-safe (load OFF in the absence of an active control signal).

10.3 Goal 3: Surplus-driven switching via an external sensor

The third goal is to verify that the controller can take its switching decision based on the value of an external sensor that represents the currently available PV surplus power. For the prototype, the surplus value can be supplied either by a real sensor (for example a current transformer or a smart-meter interface) or, for repeatable laboratory testing, by a simulated value injected via the local interface or via Home Assistant.

Test 3.1 – Sensor signal acquisition. A defined surplus value is applied to the sensor input. The internal value read by the firmware is displayed on the OLED and/or transmitted to Home Assistant. The expected result is that the displayed value matches the applied value within an acceptable tolerance.

Test 3.2 – Switch-on threshold. The mode selector is set to *AUTO* and the load is initially OFF. The surplus value is slowly increased. When the configured switch-on threshold is reached, the controller shall activate the load (verified by Test 1.3 / Test 2.2 setup). Activation shall only occur after the configured safety delay has elapsed.

Test 3.3 – Switch-off threshold and hysteresis. Starting from a switched-on state, the surplus value is reduced. The load shall remain ON until the surplus drops below the switch-off threshold (which is lower than the switch-on threshold). This confirms that the implemented hysteresis prevents rapid toggling around a single threshold.

Test 3.4 – Manual override. While the system is in *AUTO* and switching according to the sensor, the operator switches the mode selector to *MANUAL* and toggles the load. The expected result is that the manual command takes precedence over the automatic decision and that the new state is reflected on the local display and in Home Assistant.

Test 3.5 – Sensor failure / loss of surplus signal. The sensor input is intentionally disconnected or set to an invalid value. The expected result is that the

controller detects the missing or implausible value and falls back to a defined safe state (load OFF) and indicates the fault locally and via Home Assistant.

Test 3.6 – Network independence. With the surplus value provided by a local sensor, the WiFi connection to Home Assistant is intentionally interrupted. The expected result is that the local control logic continues to operate correctly. After the network is restored, the current state shall be re-synchronized with Home Assistant.

10.4 Goal 4: Standalone operation with integrated three-phase measurement

The fourth goal verifies the final expansion stage. It confirms that the controller can determine the PV surplus on its own through three integrated phase measurements and that it remains fully operational without any external system. This goal builds on Goals 1 to 3: switching is already verified, only the source of the surplus information is now provided by the device itself.

Test 4.1 – Per-phase measurement accuracy. For each of the three phases, a known reference load is applied (for example a calibrated resistive load on one phase at a time). The values for voltage, current, and active power reported by the firmware are compared against a reference instrument. The expected result is that the measured values match the reference within an acceptable tolerance for all three phases.

Test 4.2 – Detection of power flow direction. A defined load is connected to one phase, and afterwards a controlled feed-in is simulated on the same phase. The firmware shall correctly distinguish between import (consumption from grid) and export (surplus into grid) and shall report the corresponding sign of the power value. This is repeated for each phase individually.

Test 4.3 – Surplus calculation across all three phases. A combined situation is created in which different power values are present on the three phases at the same time. The firmware shall compute the total balance (sum across all three phases) and provide it as the internal surplus value used by the control logic. The expected result is that the computed total matches the externally measured total.

Test 4.4 – Switching decision based on internal measurement only. The external surplus input used in Goal 3 is intentionally disabled. The system is then operated solely on the basis of its own three-phase measurement. The expected result is that the load is switched ON and OFF correctly when the internally computed surplus crosses the configured thresholds, equivalent to the behavior verified in Tests 3.2 and 3.3.

Test 4.5 – Full standalone operation. The WiFi connection, the link to Home Assistant, and any other external interface are intentionally disabled. The controller shall continue to acquire its own measurements, evaluate the surplus, and switch the

load entirely on its own. The expected result is that the local user interface and the load behavior remain fully consistent with the configured logic.

Test 4.6 – Battery-backed operation. The low-voltage supply of the controller is switched from the mains-derived supply to a battery (or UPS) source. While running on the battery, the controller shall continue to perform its measurement and decision tasks. The expected result is that the load is switched correctly as long as the battery provides power, and that the controller transitions between mains and battery supply without resetting or losing its state.

Test 4.7 – Behavior on mains loss with battery backup. With battery backup active, the AC supply of the household feed is interrupted. The expected result is that the contactor drops out (the load is de-energized because no AC is available), while the ESP32 continues to run on the battery, monitors the situation, and indicates the fault state on the local display. After the AC supply is restored, the controller shall return to normal operation in a defined and safe manner.

10.5 Additional Verification: Temperature Monitoring

In addition to the three primary goals, the temperature of the driver stage and, if applicable, of the connected load (for example a heating element) shall be monitored during the load tests. A temperature sensor reads the relevant component temperatures, and the firmware shall switch the load OFF if a configurable temperature limit is exceeded. This test is executed during the long-duration switching cycles of Test 2.3.

10.6 Summary of the Test Sequence

The test sequence is intentionally structured so that each goal builds on the previous one. Goal 1 verifies the safe low-voltage control of an AC load, Goal 2 extends this to a real three-phase high-power load, and Goal 3 verifies that the switching decision is correctly derived from an external surplus sensor. Goal 4 closes the loop by replacing the external sensor with an integrated three-phase measurement, turning the controller into a fully standalone, optionally battery-backed device. The additional temperature monitoring ensures that the prototype behaves safely also under continuous operation. Successful completion of all listed test cases shall be considered as verification that the prototype fulfills the project goals defined in this concept, including the planned final expansion stage.